

AI휴먼 - 11일차

Pandas 데이터 조회와 전처리 — loc·iloc, 결측치, 중복, 자료형 변환



오늘의 학습 목표

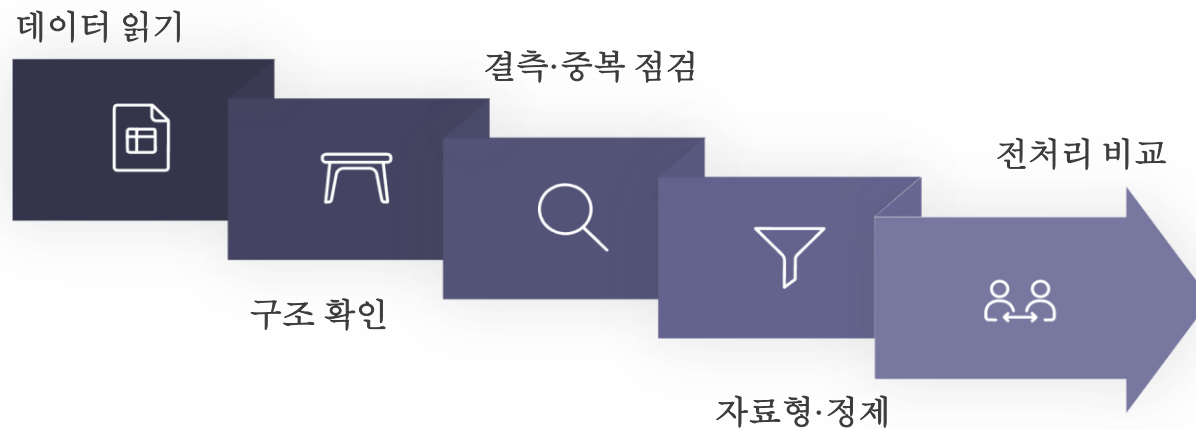
- 1 loc와 iloc의 차이를 설명하고 필요한 행과 열을 조회할 수 있습니다.
- 2 결측치(Missing Value)의 의미와 발생 원인을 설명할 수 있습니다.
- 3 isna(), fillna(), dropna()를 이용해 결측치를 확인하고 처리할 수 있습니다.
- 4 duplicated(), drop_duplicates()를 이용해 중복 데이터를 확인하고 제거할 수 있습니다.
- 5 astype()과 pd.to_numeric()을 이용해 분석에 맞는 자료형으로 변환할 수 있습니다.
- 6 전처리 전후의 행 수, 결측치 수, 자료형과 통계를 비교하여 처리 결과를 설명할 수 있습니다.

전날 학습과 오늘 학습의 연결

10일차에는 CSV 파일을 DataFrame으로 읽고 구조를 확인했습니다.

```
df = pd.read_csv("data.csv")
print(df.head())
print(df.shape)
print(df.columns)
print(df.dtypes)
df.info()
print(df.describe())
```

11일차에서는 확인한 문제를 실제로 정리합니다.



데이터 전처리란 무엇인가

데이터 전처리(Data Preprocessing)는 원본 데이터를 분석 가능한 형태로 정리하는 과정입니다.

원본 데이터에는 다음과 같은 문제가 포함될 수 있습니다.

- 값이 비어 있음
- 같은 행이 여러 번 저장됨
- 숫자가 문자열로 저장됨
- 잘못된 값이 입력됨
- 필요한 행과 열만 골라야 함

전처리의 목적은 데이터를 무조건 삭제하는 것이 아닙니다.

분석 목적에 맞는 기준을 세우고 데이터의 품질을 높이는 것이 핵심입니다.

핵심 용어 정리

용어	영문/표현	의미
데이터 전처리	Data Preprocessing	분석 전에 데이터를 정리하고 변환하는 과정
데이터 정제	Data Cleaning	오류, 결측치, 중복 등을 정리하여 데이터 품질을 높이는 작업
레이블 기반 조회	loc	행·열의 이름 또는 조건을 기준으로 데이터를 선택
위치 기반 조회	iloc	행·열의 정수 위치를 기준으로 데이터를 선택
결측치	Missing Value	값이 비어 있거나 알 수 없는 상태
NaN	Not a Number	수치 데이터에서 자주 표현되는 결측 상태
중복	Duplicate	동일하거나 중복된 의미를 가진 데이터가 반복된 상태
자료형 변환	Type Casting	데이터를 다른 자료형으로 변환하는 작업
강제 숫자 변환	pd.to_numeric()	숫자로 바꿀 수 없는 값을 처리하면서 숫자형으로 변환하는 함수

loc의 기본 개념

loc는 레이블(Label)을 기준으로 행과 열을 선택합니다.

기본 형식

```
df.loc[행_조건, 열_조건]
```

예시

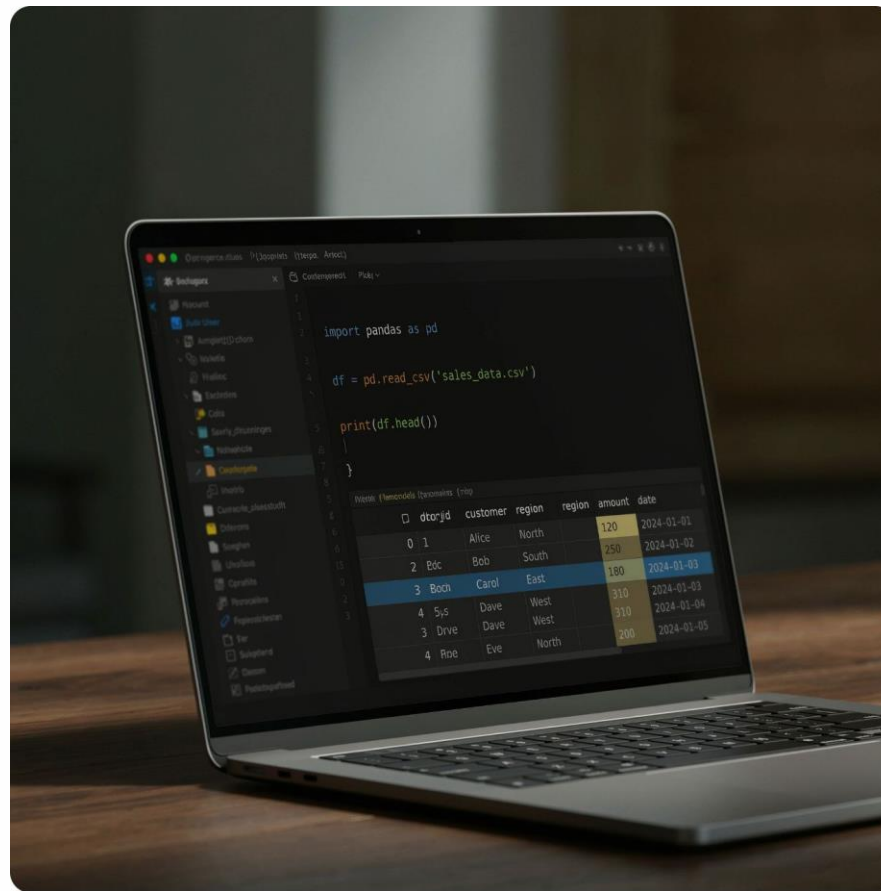
```
print(df.loc[0, "이름"])
```

여러 행과 여러 열

```
print(df.loc[0:2, ["이름", "점수"]])
```

조건 조회

```
print(df.loc[df["점수"] >= 80, ["이름", "점수"]])
```



iloc의 기본 개념

iloc는 정수 위치(Integer Position)를 기준으로 행과 열을 선택합니다.

기본 형식

```
df.iloc[행_위치, 열_위치]
```

예시

```
print(df.iloc[0, 1])
```

앞 3행과 앞 2열

```
print(df.iloc[0:3, 0:2])
```

Python 슬라이싱과 동일하게 끝 위치는 포함되지 않습니다.

 0:3 → 0, 1, 2 위치

loc와 iloc 비교

구분	loc	iloc
기준	행·열 레이블, 조건	정수 위치
컬럼 선택	"점수"	2
조건식 사용	가능	일반적으로 위치 중심
슬라이싱	끝값 레이블 범위에서는 포함	포함하지 않음
대표 사용	조건 필터링	위치 기준 일부 데이터 확인

예시

```
df.loc[df["지역"] == "서울", ["고객ID", "매출"]]  
  
df.iloc[0:5, 0:3]
```


조건 조회의 기본 패턴

단일 조건

```
result = df.loc[df["점수"] >= 80]
```

여러 조건

```
result = df.loc[
    (df["점수"] >= 80) &
    (df["지역"] == "서울")
]
```

OR 조건

```
result = df.loc[
    (df["지역"] == "서울") |
    (df["지역"] == "부산")
]
```



Pandas 조건식에서는 각 조건을 괄호로 감싸는 것이 중요합니다.

결측치란 무엇인가

결측치(Missing Value)는 값이 존재하지 않거나 알 수 없는 데이터입니다.

발생 예

- 설문 응답자가 문항을 비워 둔 경우
- 시스템에서 일부 값이 수집되지 않은 경우
- 파일 병합 중 대응되는 데이터가 없는 경우
- 잘못된 문자열을 숫자로 변환하면서 값이 유실된 경우

Pandas에서 자주 보는 결측 표현

`pd.NA`

`NaN`

`pd.NaT`

Pandas의 주요 결측 표현과 설정

항목	역할	샘플
pd.NA	범용적인 Pandas 결측값 표현	pd.Series([10, pd.NA])
pd.NaT	날짜·시간 데이터의 결측값 표현	pd.Series([pd.Timestamp("2026-01-01"), pd.NaT])
pd.options.display.max_rows	출력할 행 수 관련 표시 설정	pd.options.display.max_rows
pd.options.display.max_columns	출력할 열 수 관련 표시 설정	pd.options.display.max_columns

샘플 코드

```
import pandas as pd
data = pd.Series([100, pd.NA, 80])
print(data)
print(data.isna())
```

결측치 표현은 종류보다 어떻게 발견하고 어떤 기준으로 처리할 것인지가 더 중요합니다.

결측치 확인하기

DataFrame 전체의 결측 여부 확인

```
print(df.isna())
```

컬럼별 결측치 개수

```
print(df.isna().sum())
```

전체 결측치 개수

```
print(df.isna().sum())
```

결측치가 아닌 값 확인

```
print(df.notna())
```



분석 전에 `isna().sum()`을 확인하는 습관이 중요합니다.

결측 행 조회하기

특정 컬럼이 결측인 행을 조회할 수 있습니다.

```
missing_rows = df.loc[df["점수"].isna()]
print(missing_rows)
```

여러 컬럼 중 하나라도 결측인 행

```
missing_rows = df.loc[
    df[["점수", "지역"]].isna().any(axis=1)
]
```

결측치 처리 전략

결측치는 상황에 따라 여러 방법으로 처리할 수 있습니다.

삭제

```
df = df.dropna()
```

특정 컬럼 기준 삭제

```
df = df.dropna(subset=["고객ID"])
```

고정값 대체

```
df["담당자"] = df["담당자"].fillna("미배정")
```

평균 대체

```
df["점수"] = df["점수"].fillna(df["점수"].mean())
```

중앙값 대체

```
df["처리시간"] = df["처리시간"].fillna(df["처리시간"].median())
```

평균·중앙값·최빈값 대체의 차이

대체 기준	적합한 상황	주의점
평균	값의 분포가 비교적 균형적인 수치 데이터	큰 이상값의 영향을 받을 수 있음
중앙값	큰 값이나 작은 값의 영향이 큰 수치 데이터	데이터의 실제 값이 아닐 수 있음
최빈값	범주형 데이터 또는 가장 일반적인 값으로 대체할 때	특정 범주 비중을 더 높일 수 있음
고정값	"미입력", "미배정"처럼 의미를 보존하고 싶을 때	분석 목적에 맞는 별도 범주인지 판단 필요
삭제	결측 데이터가 적고 삭제 영향이 작을 때	데이터 손실 가능

중복 데이터란 무엇인가

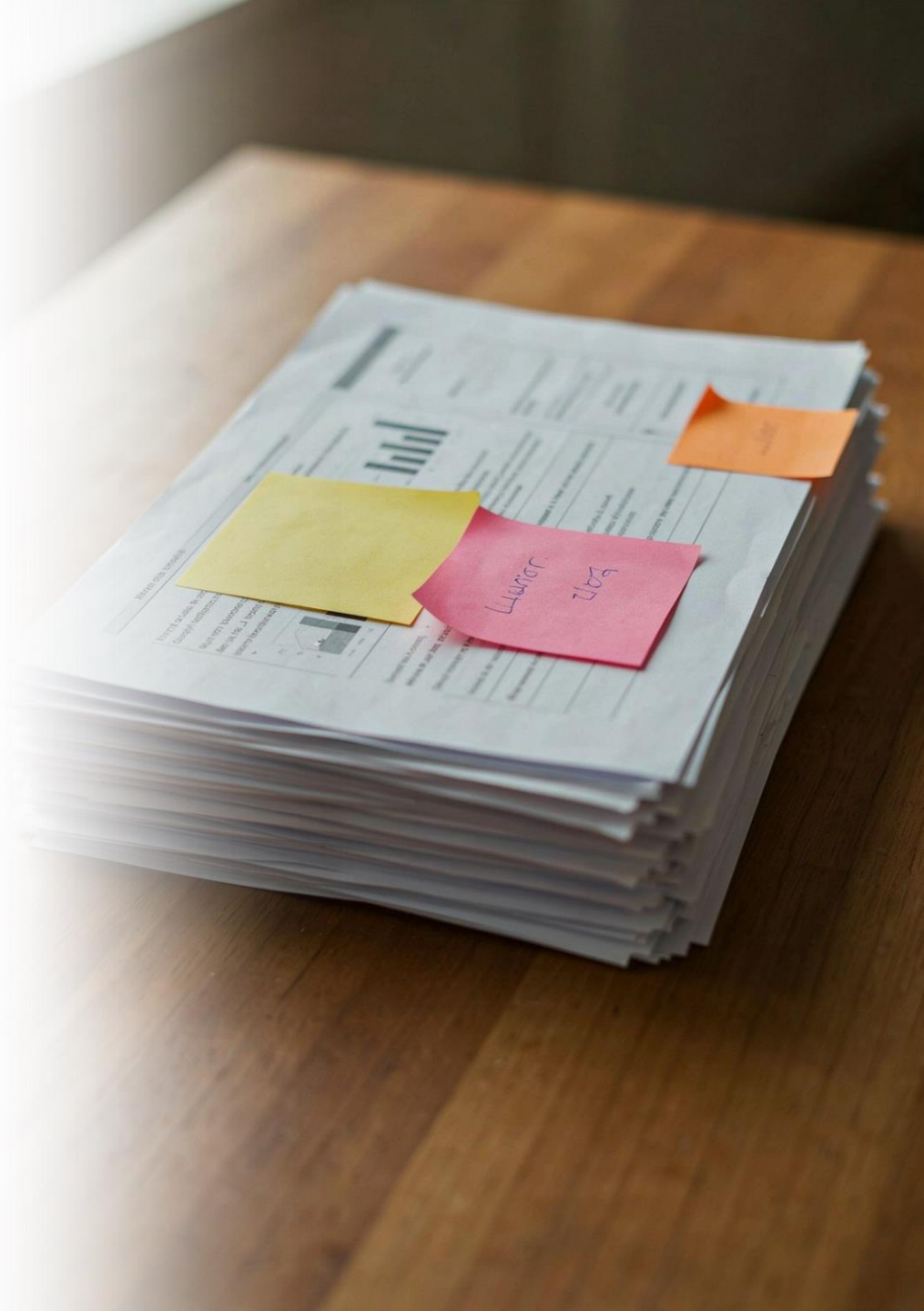
중복 데이터(Duplicated Data)는 같은 데이터가 여러 번 기록된 상태입니다.

발생 예

- 동일 파일을 두 번 합침
- 시스템 재전송
- 수작업 복사
- 동일 주문을 여러 번 저장
- 식별번호는 다르지만
실제 의미가 같은 데이터

중복을 제거하기 전에 다음을 확인해야 합니다.

- 완전히 동일한 행인가?
- 특정 컬럼만 기준으로 중복인가?
- 실제 반복 거래인가?
잘못 저장된 데이터인가?



중복 확인하기

중복 여부 확인

```
print(df.duplicated())
```

중복 행 개수 확인

```
print(df.duplicated().sum())
```

중복 행만 조회

```
print(df.loc[df.duplicated()])
```

특정 컬럼 기준 중복 확인

```
print(df.duplicated(subset=["주문번호"]))
```



중복 기준을 명확하게 설정하는 것이 중요합니다.

중복 제거하기

완전히 동일한 중복 행 제거

```
df = df.drop_duplicates()
```

특정 컬럼 기준 중복 제거

```
df = df.drop_duplicates(  
    subset=["주문번호"]  
)
```

처음 데이터 유지

```
df = df.drop_duplicates(  
    subset=["주문번호"],  
    keep="first"  
)
```

마지막 데이터 유지

```
df = df.drop_duplicates(  
    subset=["주문번호"],  
    keep="last"  
)
```

자료형이 중요한 이유

같은 값처럼 보여도 자료형이 다르면 계산 결과가 달라질 수 있습니다.

문자열끼리 더하기

```
"100" + "200"
```

결과

```
"100200"
```

숫자끼리 더하기

```
100 + 200
```

결과

```
300
```

⚠ 따라서 `df.dtypes`로 자료형을 반드시 확인해야 합니다.

astype()으로 자료형 변환하기

astype()은 컬럼을 지정한 자료형으로 변환합니다.

문자열로 변환

```
df["고객ID"] = df["고객ID"].astype("string")
```

정수로 변환

```
df["수량"] = df["수량"].astype("int64")
```

실수로 변환

```
df["점수"] = df["점수"].astype("float64")
```

주의

```
df["점수"] = df["점수"].astype("int64")
```

⚠ 컬럼에 "미응시" 같은 문자열이 있다면 오류가 발생할 수 있습니다.

pd.to_numeric()으로 안전하게 숫자 변환하기

혼합된 데이터를 숫자로 변환할 때 유용합니다.

```
df["점수"] = pd.to_numeric(  
    df["점수"],  
    errors="coerce"  
)
```

errors="coerce"의 의미

- 숫자로 변환 가능한 값 → 숫자로 변환
- 변환할 수 없는 값 → NaN

예

"80" → 80
"92.5" → 92.5
"미응시" → NaN

☐ 변환 후에는 새로 발생한 결측치를 다시 확인해야 합니다.

주요 전처리 함수·속성·메서드 정리

구분	이름	역할
인덱서	df.loc[]	레이블·조건 기준 행과 열 조회
인덱서	df.iloc[]	정수 위치 기준 행과 열 조회
메서드	isna()	결측 여부 확인
메서드	notna()	결측이 아닌 값 확인
메서드	dropna()	결측 데이터 삭제
메서드	fillna()	결측값 대체
메서드	uplicated()	중복 여부 확인
메서드	drop_duplicates()	중복 데이터 제거
메서드	astype()	지정 자료형으로 변환
함수	pd.to_numeric()	숫자형 변환
속성	dtypes	컬럼별 자료형 확인
메서드	reset_index()	Index를 다시 정리

전처리 샘플 코드

```
import pandas as pd
df = pd.read_csv("customer.csv")
print(df.isna().sum())
print("중복:", df.duplicated().sum())
df["처리시간"] = pd.to_numeric(
    df["처리시간"],
    errors="coerce"
)
df["처리시간"] = df["처리시간"].fillna(
    df["처리시간"].median()
)
df["담당자"] = df["담당자"].fillna("미배정")
df = df.drop_duplicates()
df["처리시간"] = df["처리시간"].astype("int64")
print(df.dtypes)
```

전처리 순서 의사코드

01

CSV 파일을 읽는다

02

원본 데이터의 행 수를 기록한다

03

원본 데이터의 결측치 수를 기록한다

04

원본 데이터의 중복 행 수를 기록한다

05

필요한 행과 열을 loc 또는 iloc로 확인한다

06

문제가 있는 자료형을 찾는다

07

숫자형이어야 하는 컬럼을 숫자로 변환한다

08

변환 과정에서 새로 생긴 결측치를 확인한다

09

결측치 처리 기준을 결정한다

10

중복 데이터의 의미를 확인한다

11

중복을 제거한다

12

필요하면 최종 자료형을
astype으로 정리한다

13

전처리 후 행 수와 결측치 수를
다시 확인한다

14

전처리 전후 통계를 비교한다

15

정제된 데이터를 저장한다

전처리 전후를 비교해야 하는 이유

전처리는 데이터를 변경하는 작업입니다.

따라서 처리 결과를 검증해야 합니다.

점검 항목	전처리 전	전처리 후
행 수	1,000	985
전체 결측치	45	0
중복 행	15	0
매출 dtype	object	int64
평균 매출	계산 불가	128,500

이 표가 있으면 무엇을 왜 바꾸었는지 설명하기 쉽습니다.

처리 근거를 기록하는 방법

전처리 결과만 남기면 나중에 처리 이유를 알기 어렵습니다.

간단한 처리 기록 예

항목	발견 문제	처리 방법	처리 근거
처리시간	"unknown" 포함	숫자 변환 후 중앙값 대체	처리시간 분포에서 큰 값의 영향 완화
담당자	일부 결측	"미배정" 대체	담당자가 없는 상태 자체를 보존
문의ID	동일 행 중복	중복 행 제거	동일 레코드의 반복 저장으로 판단

실무에서는 전처리 코드와 처리 근거를 함께 남기는 것이 중요합니다.

자주 발생하는 오류

상황	원인	해결 방향
KeyError	존재하지 않는 컬럼명	df.columns 확인
ValueError	문자열이 섞인 컬럼을 바로 astype(int)	pd.to_numeric(errors="coerce") 검토
조건식 오류	괄호 없이 &, 사용	각 조건을 괄호로 감싸기
예상보다 많은 행 삭제	dropna()를 전체 행에 적용	subset= 기준 검토
실제 거래를 중복으로 삭제	중복 판단 기준이 잘못됨	식별키와 업무 의미 확인
변환 뒤 NaN 증가	숫자로 바꿀 수 없는 값이 존재	변환 전 원본 값 확인

데이터 정제 체크리스트

- ☐ 원본 파일을 변경하지 않고 별도의 DataFrame으로 작업했는가?
- ☐ 행 수와 열 수를 먼저 확인했는가?
- ☐ 결측치 개수를 컬럼별로 확인했는가?
- ☐ 결측치 처리 이유를 설명할 수 있는가?
- ☐ 중복 기준을 정확히 정의했는가?
- ☐ 숫자형이어야 할 컬럼의 dtype을 확인했는가?
- ☐ `pd.to_numeric()` 이후 새 결측치를 다시 확인했는가?
- ☐ 전처리 전후 행 수를 비교했는가?
- ☐ 전처리 전후 주요 통계를 비교했는가?
- ☐ 정제된 데이터와 처리 근거를 함께 남겼는가?

오늘의 핵심 정리

loc는 레이블과 조건을 기준으로 데이터를 조회합니다.

iloc는 정수 위치를 기준으로 데이터를 조회합니다.

결측치는 다음 흐름으로 처리합니다.

확인 → 원인 판단 → 처리 기준 결정 →
처리 → 재확인

중복 데이터는 실제 반복 데이터인지 오류로 복제된 데이터인지 먼저 판단해야 합니다.

문자열이 섞인 숫자 컬럼은 `pd.to_numeric(errors="coerce")`를 활용할 수 있습니다.

`astype()`은 최종 자료형을 명확하게 정리할 때 유용합니다.

전처리 후에는 반드시 행 수, 결측치, 중복, dtype, 주요 통계를 다시 확인합니다.

다음 강의 연결

다음 강의에서는 정제된 데이터를 기준으로 실무형 요약표를 만듭니다.

연결되는 핵심 주제

groupby()를 이용한 그룹 집계

agg()를 이용한 여러 통계 계산

sort_values()를 이용한 정렬

merge()를 이용한 DataFrame 결합

pivot_table()을 이용한 교차 요약표